

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«КАЗАНСКИЙ (ПРИВОЛЖСКИЙ) ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт Вычислительной математики и информационных технологий

Отчет
учебной практики (научно-исследовательская работа (получение первичных
навыков научно-исследовательской работы))

Обучающийся Филадельфов Тихон Георгиевич гр.09-535 _____
(ФИО студента) (Группа) (Подпись)

Научный руководитель:
Доцент кафедры САИТ Васильев А.В. _____
(Подпись)

Руководитель практики от КФУ:

Доцент кафедры САИТ Андрианова А.А.

Оценка за практику _____
(Подпись)

Дата сдачи отчета « ____ » _____ 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. ПЕРЕХОД ОТ АНАЛИЗА ПРЕДМЕТНОЙ ОБЛАСТИ К РАЗРАБОТКЕ БИБЛИОТЕКИ.....	5
2. АРХИТЕКТУРА И РЕАЛИЗАЦИЯ БИБЛИОТЕКИ QUBIK.....	7
3. ДАННЫЕ, МОДЕЛИ И МЕТОДИКА ЭКСПЕРИМЕНТОВ.....	11
4. РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ.....	15
ЗАКЛЮЧЕНИЕ.....	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	25
ПРИЛОЖЕНИЯ.....	28

ВВЕДЕНИЕ

Учебная практика (научно-исследовательская работа (получение первичных навыков научно-исследовательской работы)) проходила на кафедре системного анализа и информационных технологий Института вычислительной математики и информационных технологий КФУ с 09 февраля 2026 по 06 июня 2026 года.

Тема работы связана с разработкой библиотеки для работы с квантовыми гибридными нейронными сетями. В первом семестре был выполнен анализ предметной области и существующих программных инструментов. Во втором семестре работа перешла в практическую плоскость: разработана собственная библиотека Qubik на основе PyTorch и с её помощью проведено систематическое экспериментальное исследование гибридных квантово-классических моделей.

Гибридные модели, в которых параметризованная квантовая схема встроена в классическую нейронную сеть, считаются основным способом практического применения квантовых вычислений в машинном обучении в эпоху шумных квантовых устройств промежуточного масштаба [13, 14]. Вместе с тем остаётся открытым вопрос, даёт ли квантовая часть реальный выигрыш или тот же результат обеспечивает классическая сеть сопоставимого размера. Корректный ответ требует аккуратного протокола: сравнения моделей с одинаковым числом параметров, контрольных архитектур, многократных запусков и проверки статистической значимости.

Цель работы — разработать библиотеку для построения, обучения и исследования гибридных квантово-классических нейронных сетей и с её помощью экспериментально определить, в каких условиях квантовый слой улучшает качество модели. Для достижения цели решались следующие задачи: реализовать симулятор квантовых схем и обучаемый квантовый слой, совместимые с механизмом автоматического дифференцирования PyTorch;

проверить корректность реализации сравнением с эталонной библиотекой; сформировать протокол честного сравнения моделей; провести серии экспериментов на табличных данных и изображениях, включая медицинские снимки; проанализировать влияние архитектурных параметров квантового слоя и проблему затухания градиентов.

1. Переход от анализа предметной области к разработке библиотеки

В первом семестре были рассмотрены принципы квантовых вычислений [1–5], классические нейросетевые методы [6–8] и программные средства квантового машинного обучения. Состояние системы из n кубитов описывается вектором из 2^n комплексных амплитуд; параметризованные квантовые вентили — это унитарные преобразования этого вектора, а выходом квантовой схемы служат средние значения наблюдаемых, например $\langle Z \rangle$ для каждого кубита. В гибридной модели классический энкодер преобразует признаки в углы вентиля, квантовая схема выполняет преобразование, а классический слой формирует ответ; все параметры обучаются совместно градиентными методами [14, 15].

Анализ библиотек PennyLane [21] и Qiskit Machine Learning показал, что они удобны для отдельных экспериментов, но плохо подходят для систематического исследования, требующего тысяч обучений моделей. В частности, в PennyLane вычисление градиентов по правилу сдвига параметра выполняется для каждого объекта выборки отдельно, что делает такой режим очень медленным, а инфраструктура для серийных экспериментов (конфигурации, повторные запуски, статистика) в этих библиотеках отсутствует.

По итогам анализа были сформулированы требования к собственной библиотеке: собственный батчевый симулятор, интегрированный с автоматическим дифференцированием PyTorch [22] и работающий на CPU и GPU; квантовый слой в виде стандартного модуля `nn.Module`; два способа вычисления градиентов — обратное распространение для симуляции и правило сдвига параметра, применимое на реальном квантовом оборудовании [19]; режим конечного числа измерений; проверяемость результатов сравнением с эталоном; инфраструктура воспроизводимых экспериментов.

Экспериментальная часть строилась вокруг трёх исследовательских вопросов. Первый — даёт ли квантовая голова выигрыш при малом объёме обучающих данных, в том числе на медицинских изображениях. Второй — как ведут себя градиенты с ростом числа кубитов и глубины схемы (эффект barren plateaus [16, 17]) и помогают ли стратегии инициализации [18]. Третий — насколько гибридная модель устойчива к условиям реального квантового процессора, прежде всего к конечному числу измерений.

2. Архитектура и реализация библиотеки Qubik

Библиотека Qubik написана на Python и опирается на PyTorch. Её модули разделены на ядро (описание схем, симулятор, бэкенды, квантовый слой, модели, обучение) и инфраструктуру экспериментов. Назначение модулей приведено в таблице 1, общая схема — на рисунке 1.

Таблица 1 – Модули библиотеки Qubik

Модуль	Назначение
<code>circuits.py</code>	Описание схем: вентили, кодирование данных, вариационные слои, экспорт в OpenQASM 2.0
<code>simulator.py</code>	Батчевый симулятор вектора состояния на PyTorch (CPU и GPU)
<code>backends.py</code>	Вычисление градиентов (backprop, parameter-shift), режим shots, эталонный бэкенд PennyLane
<code>layers.py</code>	QuantumLayer — обучаемый квантовый слой, модуль PyTorch
<code>models.py</code>	Классические, квантовые, гибридные и контрольные модели, варианты энкодера
<code>data.py, features.py</code>	Подготовка табличных данных, извлечение признаков изображений сетью ResNet18
<code>training.py</code>	Обучение с ранней остановкой, метрики качества
<code>experiments/</code>	Серии экспериментов по конфигурациям, анализ, barren plateaus, замер скорости

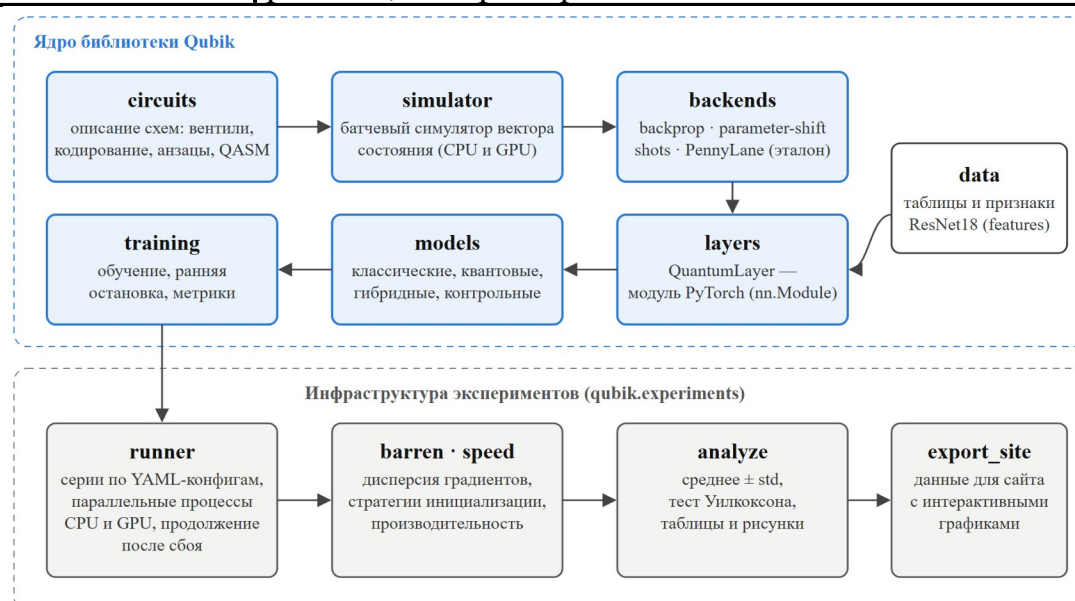


Рисунок 1 – Архитектура библиотеки Qubik

Симулятор хранит состояние батча из B объектов в виде тензора формы $(B, 2, \dots, 2)$ с 2^n комплексными амплитудами для каждого объекта. Однокубитный вентиль применяется как умножение на матрицу 2×2 вдоль оси соответствующего кубита, вентиль CNOT реализован как перестановка индексов базисных состояний, вентиль CZ — как умножение на вектор знаков. Все операции являются стандартными дифференцируемыми операциями PyTorch, поэтому градиенты по углам вентиля вычисляются штатным механизмом autograd, а схемы для всех объектов батча выполняются одновременно, в том числе на GPU. Вспомогательные тензоры кэшируются на нужном устройстве.

Квантовый слой QuantumLayer формирует углы всех параметризованных вентилях из входных признаков и обучаемых весов одной дифференцируемой операцией выборки по индексам, поэтому градиент автоматически раскладывается на составляющие по входам и по весам. Поддерживаются три способа кодирования ($RY(x)$, $RX(x)$ и плотное кодирование $H \cdot RZ \cdot RY$), три вариационных анзаца (strongly entangling, $RY + CNOT$, hardware-efficient), повторная загрузка данных [20] и три стратегии инициализации весов: равномерная на $[0, 2\pi)$, малая нормальная $N(0; 0, 1^2)$ [18] и нулевая. Выход слоя — средние значения $\langle Z \rangle$ для всех кубитов.

Кроме обратного распространения, реализовано правило сдвига параметра [19]: для вентилях вида $\exp(-i\theta P/2)$ производная равна $\partial \langle O \rangle / \partial \theta = [\langle O \rangle(\theta + \pi/2) - \langle O \rangle(\theta - \pi/2)] / 2$. Это правило даёт точный градиент и применимо на реальном квантовом процессоре. В Qubik все $2G$ сдвинутых схем (G — число параметризованных вентилях) выполняются одним батчем, что многократно ускоряет вычисление. Режим конечного числа измерений моделирует оценку $\langle Z \rangle$ по shots измерениям через биномиальное распределение.

Гибридная модель (рисунок 2) состоит из классического энкодера, квантового слоя и линейного выходного слоя. По результатам экспериментов

(раздел 4.2) в качестве основного варианта энкодера выбрана последовательность $\text{Linear} \rightarrow \text{BatchNorm} \rightarrow (\pi/2) \cdot \tanh$.

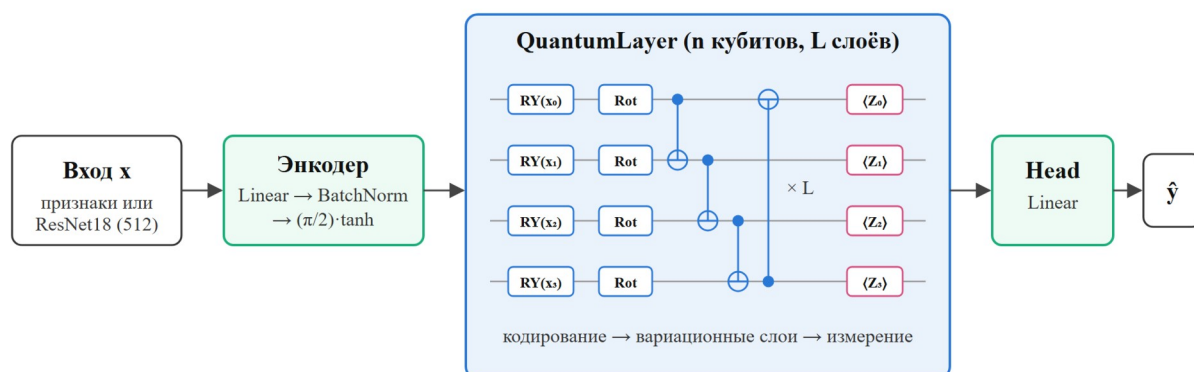


Рисунок 2 – Схема гибридной квантово-классической модели

Корректность проверяется 158 автоматическими тестами. На 96 конфигурациях схем (1–5 кубитов, разные кодирования, анзацы и глубина) результаты собственного симулятора совпадают с эталонной библиотекой PennyLane с точностью 10^{-5} ; градиенты по правилу сдвига параметра совпадают с градиентами обратного распространения; проверяются нормировка состояния, известные состояния (например, состояние Белла) и обучаемость всех моделей.

Производительность симулятора показана на рисунке 3. В режиме обратного распространения на CPU Qubik быстрее PennyLane в 1,9 раза на 4 кубитах и в 3,7 раза на 12 кубитах; в режиме правила сдвига параметра — примерно в 400 раз (68 мс против 27 с на шаг обучения для 6 кубитов) за счёт батчевого исполнения. Для схем от 14 кубитов выгодно использовать GPU: на 16 кубитах шаг на Tesla V100 в 4,6 раза быстрее, чем на CPU.

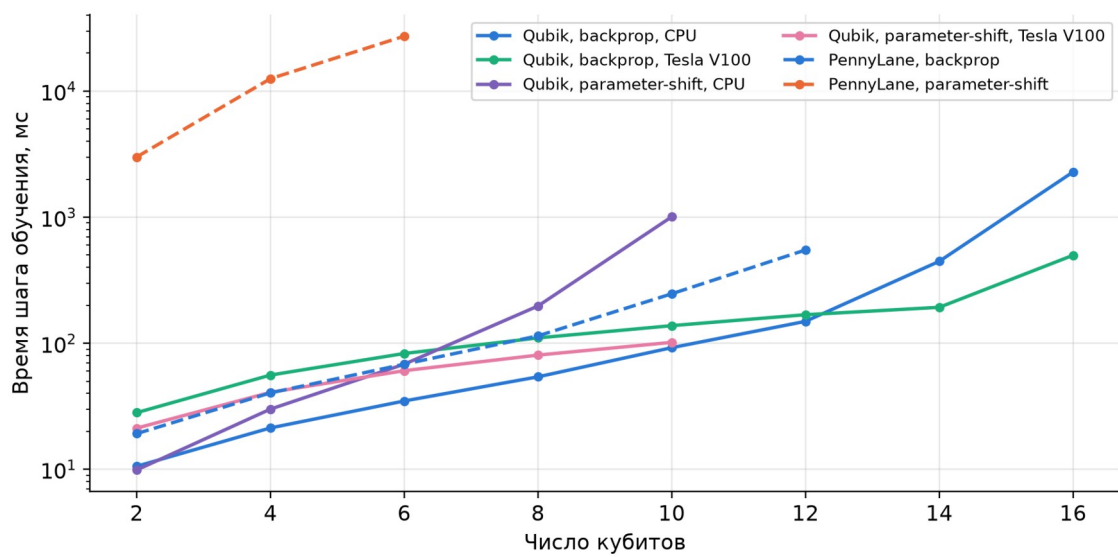


Рисунок 3 – Время шага обучения квантового слоя (батч 32, 2 слоя)

3. Данные, модели и методика экспериментов

Для экспериментов использовались открытые наборы данных (таблица 2). Табличные наборы встроены в scikit-learn [9, 10]. Изображения преобразовывались в 512 признаков замороженной сетью ResNet18 с весами ImageNet [23], после чего обучалась только голова модели — классическая, квантовая или гибридная, как в работе [15]. Медицинские наборы взяты из коллекции MedMNIST v2 [24].

Таблица 2 – Наборы данных

Набор данных	Задача	Объём	Классов
Iris, Wine	Классификация по признакам	150; 178	3
Breast Cancer Wisconsin	Диагностика опухоли	569	2
Moons, Circles	Синтетические данные	по 400	2
MNIST [25]	Рукописные цифры	10 000 / 2 000	10
Fashion-MNIST	Предметы одежды	10 000 / 2 000	10
PneumoniaMNIST [24]	Рентген грудной клетки	4 708 / 624	2
BreastMNIST [24]	УЗИ молочной железы	546 / 156	2

Сравнивались пять моделей (таблица 3). Кроме классической сети, квантовой и гибридной моделей, в сравнение включены две контрольные: многослойный перцептрон с тем же числом параметров, что у гибрида, и модель bottleneck, повторяющая архитектуру гибрида, но с классическим слоем вместо квантового. Первая отвечает на вопрос о вкладе квантового слоя при равном размере модели, вторая — о вкладе именно квантовой части при одинаковой структуре.

Таблица 3 – Модели для сравнения

Модель	Архитектура	Роль в сравнении
Classical MLP	Перцептрон 32–16, ReLU	Сильная классическая базовая модель
MLP (равное число параметров)	Один скрытый слой, размер подобран под гибрида	Сравнение при равном размере

Модель	Архитектура	Роль в сравнении
Bottleneck	Энкодер $\rightarrow$ Linear(n, n) + tanh $\rightarrow$ выходной слой	Вклад именно квантового слоя
Quantum QNN	РСА до n признаков $\rightarrow$ квантовый слой $\rightarrow$ линейное считывание	Чистый квантовый подход
Hybrid QNN	Энкодер $\rightarrow$ квантовый слой $\rightarrow$ выходной слой	Основной объект исследования

Протокол экспериментов был единым для всех серий. Табличные данные стратифицированно делились на обучающую, валидационную и тестовую выборки в пропорции 60/20/20; для изображений тестовая выборка фиксирована, а из обучающей берётся подвыборка нужного размера. Стандартизация признаков и РСА обучались только на обучающей выборке. Каждая конфигурация запускалась с 10 значениями seed, который задаёт и разбиение данных, и инициализацию модели. Обучение — оптимизатор Adam с шагом 0,01, пакет 32 объекта, до 300 эпох (150 для изображений) с ранней остановкой по функции потерь на валидации и возвратом лучших весов [7, 11]. Качество оценивалось по accuracy и macro-F1 на тестовой выборке; различия между моделями проверялись парным критерием Уилкоксона на уровне значимости 0,05 [12].

Всего проведено восемь серий экспериментов (таблица 4) — 6550 обучений. Вместе с первыми сериями, в которых использовался исходный энкодер $\pi \cdot \tanh$ (раздел 4.2), на сервере обучена 8961 модель. Вычисления выполнялись на сервере с двумя процессорами Intel Xeon E5-2698 v4 (80 потоков), 64 ГБ памяти и графическим ускорителем NVIDIA Tesla V100: до 130 процессов обучения работали параллельно, суммарное время обучения моделей составило около 364 процессорных часов.

Таблица 4 – Серии экспериментов

Серия	Что варьируется	Обучений
Табличные данные	5 моделей $\times$ 5 наборов $\times$ 2 варианта энкодера	350
Изображения	4 набора $\times$ 6 размеров выборки (50–2000) $\times$ 4 и 8 кубитов	3 120

Серия	Что варьируется	Обучений
12 кубитов на изображениях	4 набора $\times$ 3 размера выборки $\times$ 2 варианта энкодера	360
Кубиты и глубина	2–12 кубитов $\times$ 1, 2, 4 слоя $\times$ 3 набора	1 080
Абляция схемы	кодирование $\times$ анзац $\times$ повторная загрузка данных	1 080
Конечное число измерений	100, 1 000, 10 000 измерений и точный расчёт	80
Инициализация весов	uniform и small $\times$ 4 и 8 кубитов $\times$ 2 и 8 слоёв	480
Barren plateaus	2–20 кубитов $\times$ 3 анзаца $\times$ 3 инициализации	336 точек

Для прозрачности все результаты опубликованы на сайте проекта <https://tests.16vpn.uk> [26] (рисунок 4). Сайт статический: во время расчётов модуль `export_site` раз в минуту собирал сводные данные из файлов результатов и журнала загрузки сервера и передавал их на внешний веб-сервер, поэтому ход экспериментов можно было наблюдать в реальном времени, а после завершения сайт хранит итоговые данные.

На вкладке «Результаты» для каждой серии экспериментов и замеров скорости приведены интерактивные графики и таблицы: средние значения, стандартные отклонения и результаты теста Уилкоксона; переключатель энкодера позволяет сравнить варианты гибридной модели. Отдельные вкладки описывают устройство библиотеки, эффект насыщения энкодера (раздел 4.2) и загрузку процессоров и графического ускорителя на протяжении всего прогона.

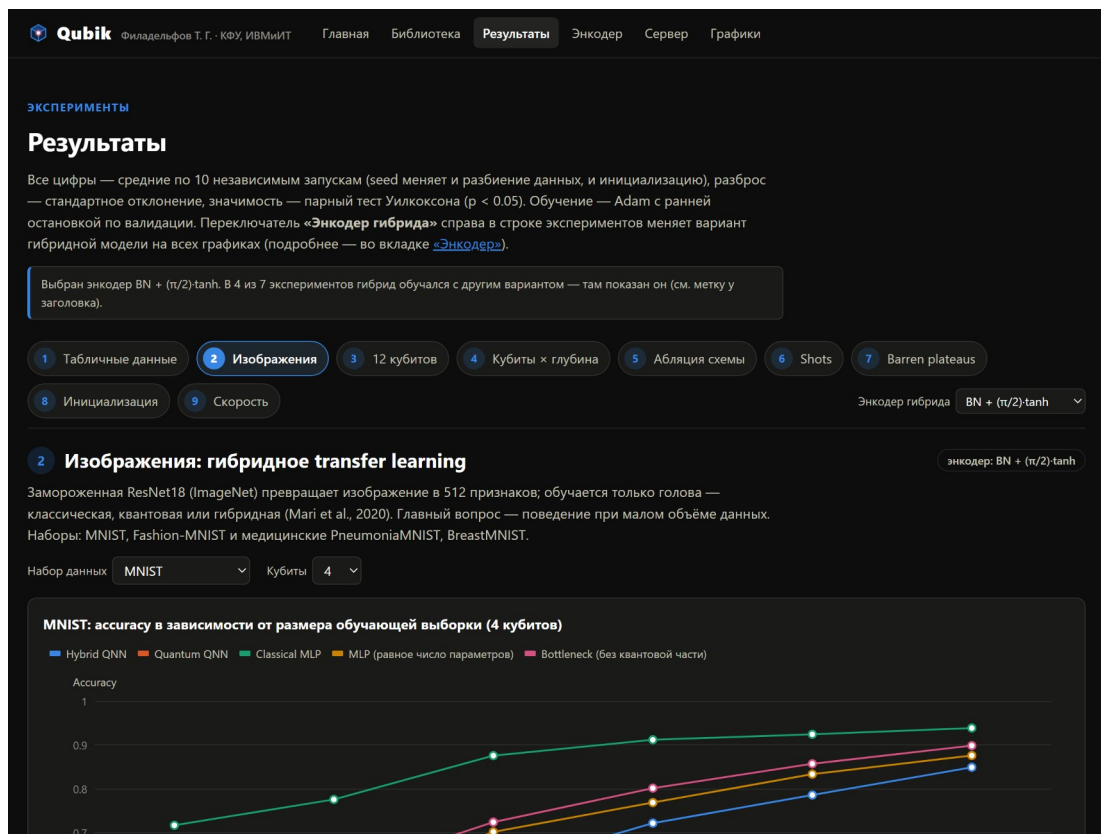


Рисунок 4 – Вкладка «Результаты» на сайте проекта

4. Результаты экспериментов

4.1 Табличные данные

Результаты на табличных данных приведены в таблице 5 и на рисунке 5. Гибридная модель значительно точнее чистой квантовой сети на наборах Breast Cancer (+8,2 п. п.), Iris (+5,7 п. п.) и Wine (+8,1 п. п.; во всех случаях $p \leq 0,004$): классический энкодер снимает ограничение чистой квантовой модели, у которой число признаков не может превышать число кубитов.

Таблица 5 – Аккуратность на табличных данных (среднее по 10 запускам)

Набор	Classical MLP	MLP (равн.)	Bottleneck	Quantum	Hybrid
Iris	0,957	0,967	0,980	0,927	0,983
Wine	0,981	0,975	0,975	0,897	0,978
Breast Cancer	0,976	0,976	0,979	0,895	0,976
Moons	0,930	0,934	0,939	0,924	0,910
Circles	0,995	0,994	0,995	0,996	0,996

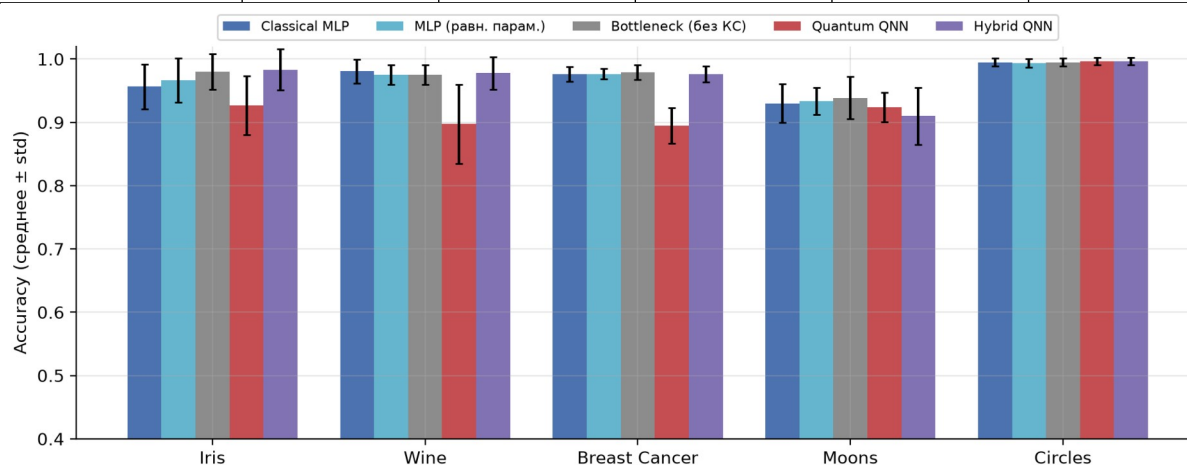


Рисунок 5 – Аккуратность пяти моделей на табличных данных (среднее ± стандартное отклонение)

При этом гибридная модель не превосходит многослойный перцептрон того же размера ни на одном наборе: различия лежат в пределах разброса и статистически незначимы. Значимыми оказались лишь выигрыш у более крупного перцептрона 32–16 на наборе Iris (+2,7 п. п., $p = 0,016$) и проигрыш модели bottleneck на наборе Moons (–2,9 п. п., $p = 0,03$). Цена квантового слоя — время: эпоха обучения гибридной модели в среднем примерно в 100 раз

дольше, чем у перцептрона того же размера, так как схема симулируется на классическом оборудовании.

4.2 Влияние энкодера на работу квантового слоя

В первой версии гибридной модели энкодер отображал признаки в углы вентилей функцией $\pi \cdot \tanh$. На изображениях (512 признаков) такая модель обучалась до уровня случайного угадывания: ассурасу 13,8 % на MNIST и 11,7 % на Fashion-MNIST, а на PneumoniaMNIST модель предсказывала только один класс (рисунок 6). Анализ показал, что оптимизатор быстро увеличивает веса энкодера, функция $\tanh$ насыщается и углы стремятся к $\pm\pi$. Но состояния $R_Y(\pi)|0\rangle$ и $R_Y(-\pi)|0\rangle$ различаются лишь глобальной фазой и неразличимы при измерении, поэтому информация о входе теряется. Модель bottleneck с тем же энкодером от насыщения почти не страдает, так как знак сигнала для неё сохраняется.

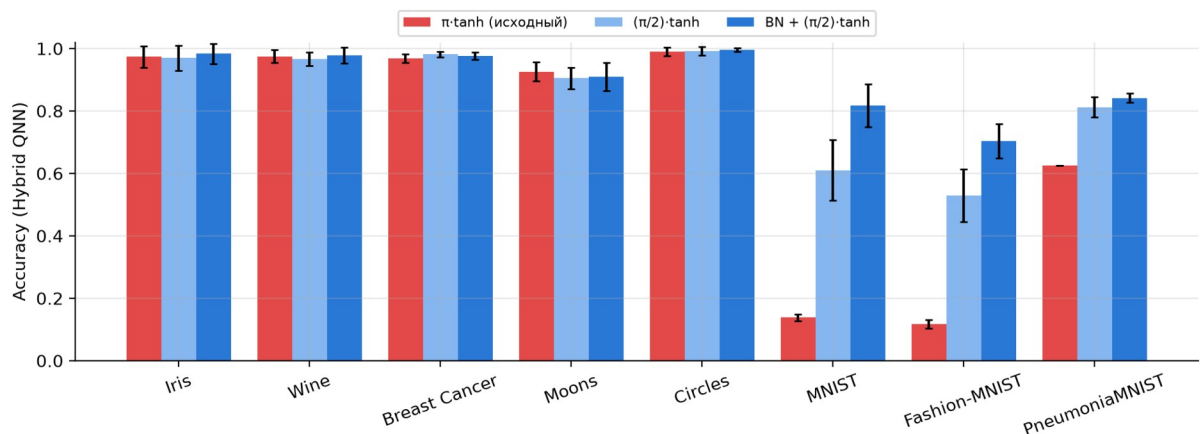


Рисунок 6 – Ассурасу гибридной модели при разных вариантах энкодера (обучающая выборка от 1000 объектов)

Замена на $(\pi/2) \cdot \tanh$, как в работе [15], устраняет неразличимость крайних значений (61,0 % на MNIST), а добавление пакетной нормализации перед $\tanh$ удерживает углы в рабочем диапазоне на протяжении всего обучения (81,7 % на MNIST, 70,4 % на Fashion-MNIST, 84,2 % на PneumoniaMNIST). Этот результат важен методически: неудачный масштаб углов кодирования систематически и незаметно занижает качество гибридной

модели и может привести к ложному выводу о бесполезности квантового слоя. Все дальнейшие результаты для изображений приведены для энкодера с пакетной нормализацией.

4.3 Изображения и малые обучающие выборки

Зависимость качества от размера обучающей выборки для головы на 4 кубитах показана на рисунке 7, основные значения — в таблице 6. Гибридная модель значительно превосходит чистую квантовую сеть на MNIST, Fashion-MNIST и PneumoniaMNIST при большинстве размеров выборки (до +33 п. п. на MNIST), но уступает классическим моделям, особенно при малом числе примеров. Гипотеза о преимуществе квантовой головы при малых выборках в данной постановке не подтвердилась.

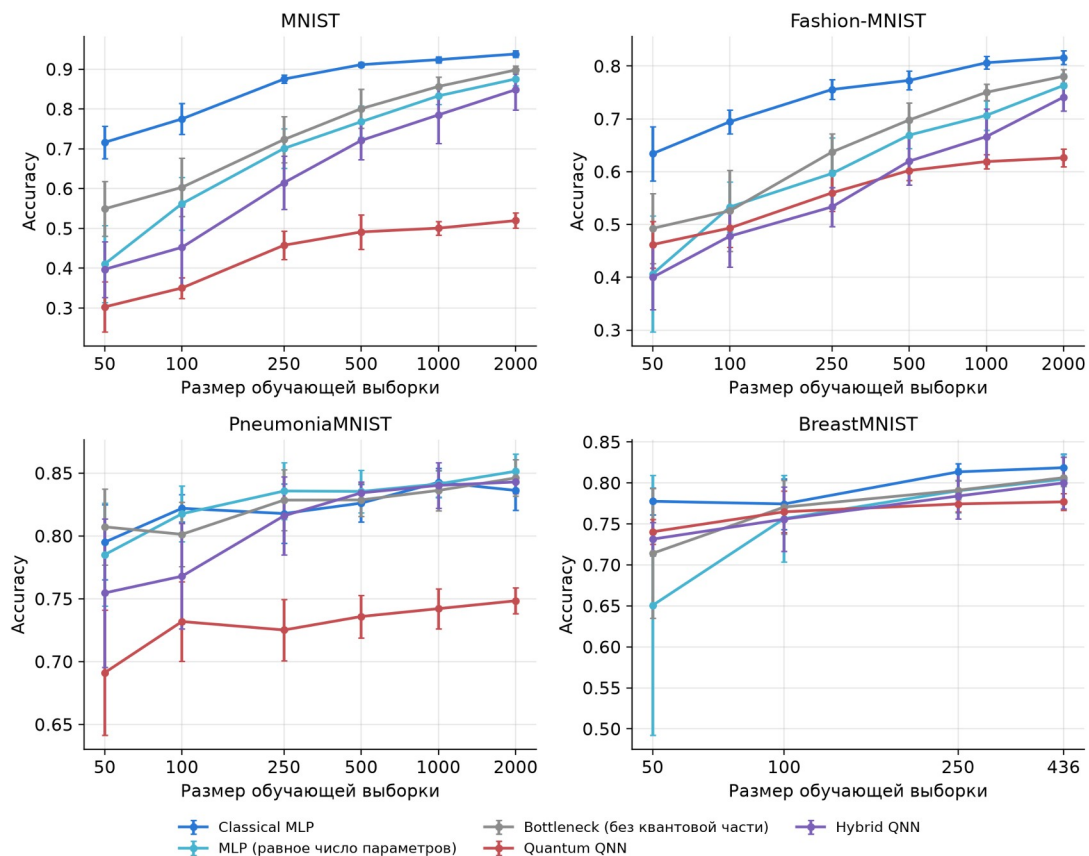


Рисунок 7 – Ассигуру моделей на изображениях в зависимости от размера обучающей выборки (4 кубита)

Таблица 6 – Accuracy на изображениях (4 кубита, энкодер с BatchNorm)

Набор, объём выборки	Classical MLP	MLP (равн.)	Bottleneck	Quantum	Hybrid
MNIST, 100	0,775	0,562	0,603	0,350	0,452
MNIST, 2000	0,938	0,875	0,898	0,520	0,848
Fashion-MNIST, 100	0,695	0,533	0,526	0,493	0,478
Fashion-MNIST, 2000	0,816	0,763	0,781	0,626	0,741
PneumoniaMNIST, 100	0,822	0,818	0,801	0,732	0,768
PneumoniaMNIST, 500	0,826	0,835	0,829	0,736	0,834
BreastMNIST, 100	0,774	0,756	0,771	0,765	0,756
BreastMNIST, 436	0,819	0,804	0,806	0,777	0,800

Разрыв с классическими моделями сокращается с ростом выборки и минимален на медицинских данных: на PneumoniaMNIST при 500 примерах гибрид (0,834) равен перцептрону того же размера (0,835), на BreastMNIST при полной выборке — модели bottleneck (0,800 и 0,806). Увеличение квантовой головы до 12 кубитов не дало систематического улучшения: на MNIST при 2000 примерах точность составила 0,890 против 0,912 для 8 кубитов, на PneumoniaMNIST — 0,843 при 0,838 у перцептрона равного размера.

4.4 Число кубитов, глубина и архитектура схемы

Результаты по сетке «число кубитов × глубина» приведены на рисунке 8. Точность гибридной модели почти не зависит от параметров схемы (0,95–0,96 в среднем по наборам), тогда как чистая квантовая сеть чувствительна к глубине: при одном слое её точность 0,88–0,89, при четырёх — 0,92–0,95. Время обучения растёт экспоненциально с числом кубитов: эпоха гибридной модели занимает 0,26 с для 2 кубитов и 11,2 с для 12 кубитов.

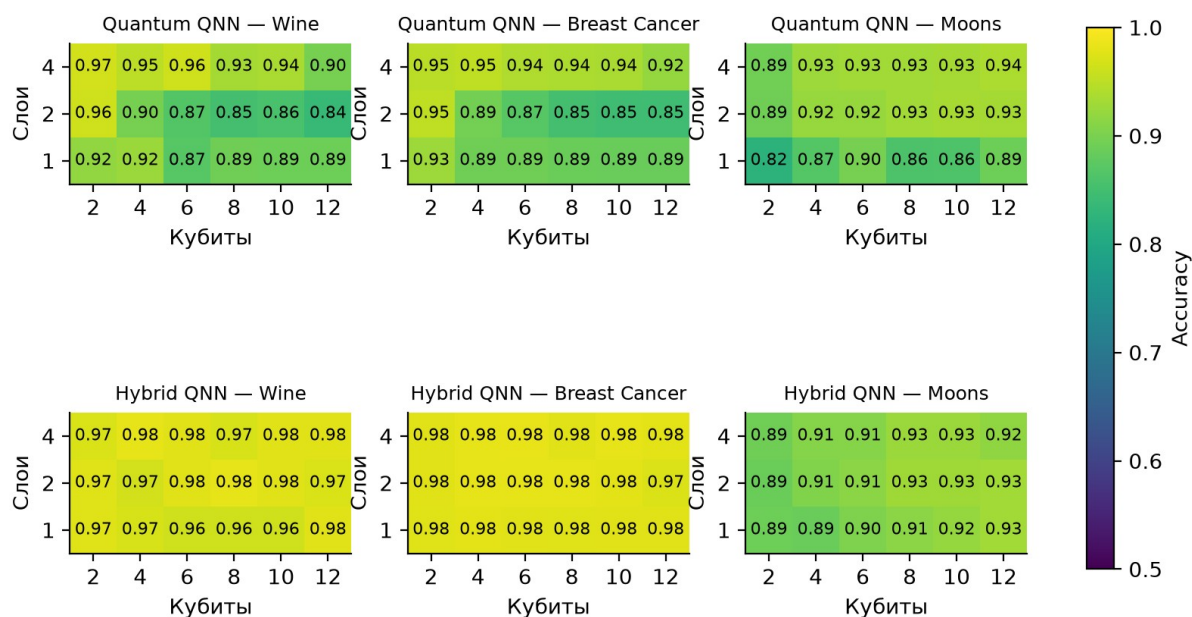


Рисунок 8 – Accuracy квантовой и гибридной моделей в зависимости от числа кубитов и числа слоёв

Абляция компонентов схемы (таблица 7) подтверждает эту закономерность. Для гибридной модели различия между способами кодирования, анзацами и режимами повторной загрузки не превышают 0,4 п. п. Для чистой квантовой сети выбор анзаца критичен (0,849 для простейшего анзаца RY + CNOT против 0,938 для strongly entangling), а кодирование через RX уступает кодированию через RY (0,872 и 0,933). Гибридная модель устойчива и к конечному числу измерений: при 100 измерениях на Breast Cancer точность снижается лишь с 0,982 до 0,974, на Moons — с 0,903 до 0,894. Таблица 7 – Абляция квантового слоя (accuracy, среднее по наборам Moons, Circles, Breast Cancer)

Фактор	Вариант	Hybrid QNN	Quantum QNN
Кодирование	RY(x) / RX(x) / H·RZ·RY	0,960 / 0,961 / 0,964	0,933 / 0,872 / 0,918
Анзац	RY + CNOT / HEA / strongly entangling	0,963 / 0,960 / 0,962	0,849 / 0,935 / 0,938
Повторная загрузка	нет / да	0,960 / 0,963	0,912 / 0,904

4.5 Barren plateaus и инициализация весов

Дисперсия производной функции стоимости по параметру схемы при случайной инициализации оценивалась по 1000 наборам параметров для схем до 20 кубитов (рисунок 9). Для глобальной функции стоимости $\langle Z_0 \dots Z_{n-1} \rangle$ дисперсия убывает экспоненциально: при 20 слоях она падает с 0,13 для 2 кубитов до $4,4 \cdot 10^{-7}$ для 20 кубитов, то есть примерно в 300 000 раз, что подтверждает эффект barren plateaus [16]. Для локальной функции $\langle Z_0 Z_1 \rangle$ в hardware-efficient анзаце дисперсия стабилизируется на уровне около 10^{-2} , что согласуется с теорией [17]; для strongly entangling анзаца затухание наблюдается и для локальной функции.

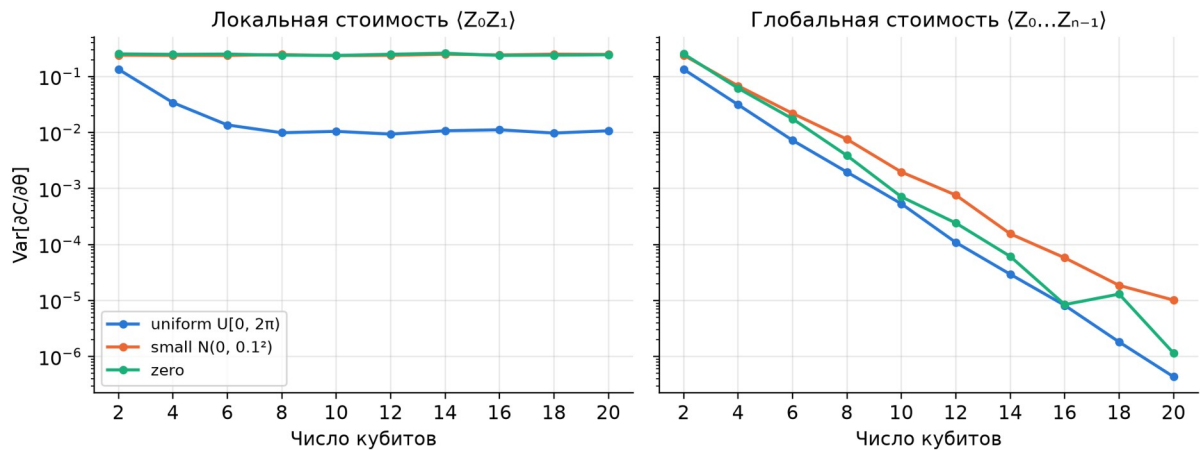


Рисунок 9 – Дисперсия градиента при разных стратегиях инициализации (НЕА, 20 слоёв)

Малая и нулевая инициализация [18] полностью сохраняют градиенты локальной функции стоимости (дисперсия около 0,25 при любом числе кубитов) и в 23 раза увеличивают дисперсию для глобальной функции на 20 кубитах, хотя экспоненциальное затухание в этом случае сохраняется. На схемах до 8 кубитов, используемых при обучении, стратегия инициализации не влияет на итоговую точность: различия не превышают 0,6 п. п.

4.6 Обобщение результатов

Проведённые эксперименты показывают, что гибридная архитектура оправдана по сравнению с чистой квантовой сетью, устойчива к параметрам

схемы и конечному числу измерений, однако на рассмотренных задачах не превосходит классические модели того же размера и значительно дороже в обучении. Наиболее перспективными направлениями дальнейшей работы являются медицинские изображения, где разрыв минимален, и методы борьбы с затуханием градиентов, эффективность которых подтверждена для локальных функций стоимости.

ЗАКЛЮЧЕНИЕ

В ходе учебной практики была продолжена работа над темой, связанной с разработкой библиотеки для работы с квантовыми гибридными нейронными сетями. Если в первом семестре основное внимание уделялось анализу предметной области и существующих решений, то во втором семестре выполнена практическая часть: разработана библиотека Qubik и проведено экспериментальное исследование гибридных моделей.

Библиотека включает собственный батчевый симулятор квантовых схем на PyTorch, обучаемый квантовый слой с поддержкой трёх способов кодирования, трёх анзацев и трёх стратегий инициализации, вычисление градиентов методом обратного распространения и по правилу сдвига параметра, режим конечного числа измерений, экспорт схем в OpenQASM и инфраструктуру воспроизводимых экспериментов. Корректность подтверждена 158 автоматическими тестами, включая сравнение с библиотекой PennyLane; по скорости в режиме правила сдвига параметра библиотека превосходит PennyLane примерно в 400 раз.

Проведено восемь серий экспериментов — 6550 обучений моделей и 336 точек исследования градиентов. Гибридная модель значимо точнее чистой квантовой сети, устойчива к архитектуре схемы и числу измерений, но не превосходит классические модели равного размера. Гипотеза о преимуществе квантовой головы при малых выборках не подтвердилась; наименьший разрыв с классическими моделями наблюдается на медицинских изображениях. Выявлен эффект насыщения энкодера, при котором квантовый слой перестаёт передавать информацию о входе, и предложен способ его устранения. Подтверждён эффект barren plateaus до 20 кубитов и показано, что инициализация, близкая к тождественной, сохраняет градиенты локальной функции стоимости. Все результаты с интерактивными графиками и журналом загрузки сервера опубликованы на сайте проекта.

Дальнейшее развитие работы связано с добавлением моделей шума квантовых устройств, запуском обученных схем на реальном квантовом процессоре, расширением экспериментов на медицинских изображениях и исследованием методов борьбы с затуханием градиентов, а также с подготовкой документации и выпуском библиотеки. Полученные результаты служат основой для магистерской диссертации.

За время практики были реализованы следующие компетенции (таблица 8):

Таблица 8 – Компетенции

Шифр компетенции	Расшифровка приобретаемой компетенции	Расшифровка освоения компетенции
УК-1	Способен осуществлять критический анализ проблемных ситуаций на основе системного подхода, вырабатывать стратегию действий	Проведён анализ существующих библиотек квантового машинного обучения и сформулированы требования к собственной библиотеке. Выработан протокол честного сравнения моделей; выявлена и устранена причина отказа гибридной модели на изображениях.
УК-4	Способен применять современные коммуникативные технологии, в том числе на иностранном(ых) языке(ах), для академического и профессионального взаимодействия	Изучены англоязычные статьи по квантовому машинному обучению и документация PyTorch, PennyLane и scikit-learn; результаты оформлены в виде отчёта, статьи и сайта проекта с интерактивными графиками.
УК-6	Способен определять и реализовывать приоритеты собственной деятельности и способы ее совершенствования на основе самооценки	Спланированы этапы работы: проектирование и реализация библиотеки, проверка корректности, серии экспериментов, анализ результатов. Сформулированы направления дальнейшего развития работы в рамках магистерской диссертации.
ОПК-1	Способен находить, формулировать и решать актуальные проблемы прикладной математики, фундаментальной информатики и информационных технологий	Поставлена задача определения условий, при которых квантовый слой улучшает качество нейронной сети; сформулированы исследовательские вопросы о малых выборках, затухании градиентов и устойчивости к измерениям.

Шифр компетенции	Расшифровка приобретаемой компетенции	Расшифровка освоения компетенции
ОПК-2	Способен применять компьютерные/суперкомпьютерные методы, современное программное обеспечение (в том числе отечественного производства) для решения задач профессиональной деятельности	Разработана библиотека на Python и PyTorch; эксперименты выполнены на многопроцессорном сервере с графическим ускорителем Tesla V100 с параллельным запуском до 130 процессов.
ОПК-3	Способен проводить анализ математических моделей, создавать инновационные методы решения прикладных задач профессиональной деятельности в области информатики и математического моделирования	Проведён сравнительный анализ пяти моделей на девяти наборах данных со статистической проверкой; исследованы зависимость от параметров схемы, эффект barren plateaus и стратегии инициализации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Нильсен, М. А. Квантовые вычисления и квантовая информация / М. А. Нильсен, И. Л. Чанг ; пер. с англ. — Москва : Мир, 2006. — 824 с.
- 2) Китаев, А. Классические и квантовые вычисления / А. Китаев, А. Шень, М. Вялый. — Москва : МЦНМО, ЧеРо, 1999. — 192 с.
- 3) Валиев, К. А. Квантовые компьютеры: надежды и реальность / К. А. Валиев, А. А. Кокин. — Ижевск : НИЦ «Регулярная и хаотическая динамика», 2001. — 352 с.
- 4) Прескилл, Дж. Квантовая информация и квантовые вычисления. Т. 1 / Дж. Прескилл ; пер. с англ. — Москва ; Ижевск : НИЦ «Регулярная и хаотическая динамика», 2008. — 464 с.
- 5) Прилипко, В. К. Физические основы квантовых вычислений. Динамика кубита : монография / В. К. Прилипко, И. И. Коваленко. — Санкт-Петербург : Лань, 2024. — 216 с.
- 6) Хайкин, С. Нейронные сети: полный курс / С. Хайкин ; пер. с англ. — 2-е изд. — Москва : Вильямс, 2006. — 1104 с.
- 7) Гудфеллоу, Я. Глубокое обучение / Я. Гудфеллоу, И. Бенджио, А. Курвилль ; пер. с англ. — Москва : ДМК Пресс, 2018. — 652 с.
- 8) Николенко, С. Глубокое обучение. Погружение в мир нейронных сетей / С. Николенко, А. Кадурын, Е. Архангельская. — Санкт-Петербург : Питер, 2018. — 480 с.
- 9) Жерон, О. Прикладное машинное обучение с помощью Scikit-Learn и TensorFlow / О. Жерон ; пер. с англ. — Санкт-Петербург : Альфа-книга, 2018. — 688 с.
- 10) Рашка, С. Python и машинное обучение / С. Рашка ; пер. с англ. — Москва : ДМК Пресс, 2017. — 418 с.
- 11) Шолле, Ф. Глубокое обучение на Python / Ф. Шолле ; пер. с англ. — Санкт-Петербург : Питер, 2018. — 400 с.

- 12) Кобзарь, А. И. Прикладная математическая статистика. Для инженеров и научных работников / А. И. Кобзарь. — Москва : Физматлит, 2006. — 816 с.
- 13) Preskill, J. Quantum computing in the NISQ era and beyond / J. Preskill // Quantum. — 2018. — Vol. 2. — Art. 79.
- 14) Schuld, M. An introduction to quantum machine learning / M. Schuld, I. Sinayskiy, F. Petruccione // Contemporary Physics. — 2015. — Vol. 56, № 2. — P. 172–185.
- 15) Mari, A. Transfer learning in hybrid classical-quantum neural networks / A. Mari, T. R. Bromley, J. Izaac [et al.] // Quantum. — 2020. — Vol. 4. — Art. 340.
- 16) McClean, J. R. Barren plateaus in quantum neural network training landscapes / J. R. McClean, S. Boixo, V. N. Smelyanskiy [et al.] // Nature Communications. — 2018. — Vol. 9. — Art. 4812.
- 17) Cerezo, M. Cost function dependent barren plateaus in shallow parametrized quantum circuits / M. Cerezo, A. Sone, T. Volkoff [et al.] // Nature Communications. — 2021. — Vol. 12. — Art. 1791.
- 18) Grant, E. An initialization strategy for addressing barren plateaus in parametrized quantum circuits / E. Grant, L. Wossnig, M. Ostaszewski, M. Benedetti // Quantum. — 2019. — Vol. 3. — Art. 214.
- 19) Schuld, M. Evaluating analytic gradients on quantum hardware / M. Schuld, V. Bergholm, C. Gogolin [et al.] // Physical Review A. — 2019. — Vol. 99. — Art. 032331.
- 20) Pérez-Salinas, A. Data re-uploading for a universal quantum classifier / A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, J. I. Latorre // Quantum. — 2020. — Vol. 4. — Art. 226.
- 21) Bergholm, V. PennyLane: Automatic differentiation of hybrid quantum-classical computations [Электронный ресурс] / V. Bergholm [et al.]. — URL: <https://arxiv.org/abs/1811.04968> (дата обращения: 20.09.2026).

- 22) Paszke, A. PyTorch: An imperative style, high-performance deep learning library / A. Paszke [et al.] // Advances in Neural Information Processing Systems. — 2019. — Vol. 32.
- 23) He, K. Deep residual learning for image recognition / K. He, X. Zhang, S. Ren, J. Sun // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — 2016. — P. 770–778.
- 24) Yang, J. MedMNIST v2 — A large-scale lightweight benchmark for 2D and 3D biomedical image classification / J. Yang, R. Shi, D. Wei [et al.] // Scientific Data. — 2023. — Vol. 10. — Art. 41.
- 25) LeCun, Y. Gradient-based learning applied to document recognition / Y. LeCun, L. Bottou, Y. Bengio, P. Haffner // Proceedings of the IEEE. — 1998. — Vol. 86, № 11. — P. 2278–2324.
- 26) Qubik: гибридные квантово-классические нейронные сети [Электронный ресурс] : сайт проекта / Т. Г. Филадельфов. — URL: <https://tests.16vpn.uk> (дата обращения: 22.09.2026).

ПРИЛОЖЕНИЯ

Приложение 1

БИБЛИОТЕКА QUBIK ДЛЯ ИССЛЕДОВАНИЯ ГИБРИДНЫХ КВАНТОВО-КЛАССИЧЕСКИХ НЕЙРОННЫХ СЕТЕЙ

Филадельфов Т. Г.

*Научный руководитель канд. физ.-мат. наук, доцент, заведующий кафедрой САИТ
Васильев А. В.*

*Казанский (Приволжский) федеральный университет, Институт вычислительной
математики и информационных технологий*

Ipod050505@gmail.com

ВВЕДЕНИЕ

Гибридные квантово-классические нейронные сети объединяют классические слои и параметризованные квантовые схемы и считаются основным способом применения квантовых вычислений в машинном обучении на современных устройствах [1, 2]. Вопрос о том, даёт ли квантовая часть выигрыш по сравнению с классической моделью того же размера, остаётся открытым. В работе представлена библиотека Qubik и результаты систематического сравнения гибридных, квантовых и классических моделей.

БИБЛИОТЕКА QUBIK

Библиотека реализована на PyTorch и содержит собственный батчевый симулятор вектора состояния, обучаемый квантовый слой QuantumLayer в виде модуля nn.Module, вычисление градиентов методом обратного распространения и по правилу сдвига параметра [4], режим конечного числа измерений и инфраструктуру воспроизводимых экспериментов. Корректность подтверждена сравнением с библиотекой PennyLane [5] на 96 конфигурациях схем. За счёт батчевого исполнения сдвинутых схем вычисление градиентов по правилу сдвига параметра выполняется примерно в 400 раз быстрее, чем в PennyLane.

МЕТОДИКА ЭКСПЕРИМЕНТОВ

Сравнивались пять моделей:

- классический многослойный перцептрон;
- перцептрон с тем же числом параметров, что у гибридной модели;
- модель bottleneck — гибрид с классическим слоем вместо квантового;
- квантовая нейронная сеть;

— гибридная модель «энкодер — квантовый слой — выходной слой».

Использовались пять табличных наборов данных и признаки изображений MNIST, Fashion-MNIST, PneumoniaMNIST и BreastMNIST, извлечённые сетью ResNet18 [3]. Каждая конфигурация обучалась с 10 значениями seed, различия проверялись критерием Уилкоксона. Всего выполнено 6550 обучений.

РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ

Выявлено, что энкодер с отображением $\pi \cdot \tanh$ при насыщении переводит входы в углы $\pm\pi$, дающие неразличимые квантовые состояния; на изображениях такая модель обучается до уровня случайного угадывания. Энкодер с пакетной нормализацией и отображением $(\pi/2) \cdot \tanh$ устраняет эффект (таблица 1).

Таблица 1 – Ассурасу гибридной модели при разных энкодерах

Набор данных	$\pi \cdot \tanh$	$(\pi/2) \cdot \tanh$	BN + $(\pi/2) \cdot \tanh$
MNIST	0,138	0,610	0,817
Fashion-MNIST	0,117	0,529	0,704
PneumoniaMNIST	0,625	0,812	0,842
Breast Cancer (табл.)	0,968	0,982	0,976

С исправленным энкодером гибридная модель значительно превосходит квантовую сеть (до +33 п. п.), устойчива к числу кубитов, глубине схемы и числу измерений, но не превосходит классические модели равного размера; наименьший разрыв наблюдается на медицинских изображениях. Эффект barren plateaus подтверждён до 20 кубитов; инициализация, близкая к тождественной, сохраняет градиенты локальной функции стоимости.

ЗАКЛЮЧЕНИЕ

Разработанная библиотека позволяет проводить воспроизводимые исследования гибридных моделей в масштабе тысяч обучений. Полученные результаты указывают, что для корректного сравнения важны модели равного размера и выбор масштаба углов кодирования. Дальнейшая работа связана с моделями шума, запуском на реальном квантовом процессоре и медицинскими изображениями.

ЛИТЕРАТУРА

- 1) Нильсен, М. А. Квантовые вычисления и квантовая информация / М. А. Нильсен, И. Л. Чанг ; пер. с англ. — Москва : Мир, 2006. — 824 с.

- 2) Preskill, J. Quantum computing in the NISQ era and beyond / J. Preskill // Quantum. — 2018. — Vol. 2. — Art. 79.
- 3) Mari, A. Transfer learning in hybrid classical-quantum neural networks / A. Mari [et al.] // Quantum. — 2020. — Vol. 4. — Art. 340.
- 4) Schuld, M. Evaluating analytic gradients on quantum hardware / M. Schuld [et al.] // Physical Review A. — 2019. — Vol. 99. — Art. 032331.
- 5) Bergholm, V. PennyLane: Automatic differentiation of hybrid quantum-classical computations [Электронный ресурс] / V. Bergholm [et al.]. — URL: <https://arxiv.org/abs/1811.04968> (дата обращения: 20.09.2026).
- 6) McClean, J. R. Barren plateaus in quantum neural network training landscapes / J. R. McClean [et al.] // Nature Communications. — 2018. — Vol. 9. — Art. 4812.

Приложение 2

Ядро симулятора вектора состояния (qubik/simulator.py):

```
def apply_1q(state: torch.Tensor, mat: torch.Tensor, q: int) -> torch.Tensor:
    """state: (B, 2, ..., 2); mat: (B, 2, 2) или (2, 2)."""
    psi = state.movedim(q + 1, -1)
    if mat.dim() == 2:
        psi = psi @ mat.T
    else:
        shape = psi.shape
        psi = psi.reshape(shape[0], -1, 2) @ mat.transpose(-1, -2)
        psi = psi.reshape(shape)
    return psi.movedim(-1, q + 1)

def apply_perm(state: torch.Tensor, perm: torch.Tensor, n: int) ->
torch.Tensor:
    flat = state.reshape(state.shape[0], -1)
    return flat[:, perm].reshape(state.shape)

def run_circuit(circuit: Circuit, angles: torch.Tensor) -> torch.Tensor:
    """Исполняет схему для батча углов (B, G), возвращает состояние (B,
    2**n)."""
    n = circuit.n_qubits
    batch = angles.shape[0]
    state = torch.zeros(batch, 2**n, dtype=CDTYPE, device=angles.device)
    state[:, 0] = 1.0
    state = state.reshape(batch, *[2] * n)
    dev = angles.device
    k = 0
    for g in circuit.gates:
        if g.name in PARAM_GATES:
            state = apply_1q(state, GATE_FN[g.name](angles[:, k]),
g.wires[0])
            k += 1
        elif g.name == "H":
            state = apply_1q(state, _h_mat(dev), g.wires[0])
        elif g.name == "CNOT":
            state = apply_perm(state, _cnot_perm(n, *g.wires, dev), n)
        elif g.name == "CZ":
            state = apply_sign(state, _cz_sign(n, *g.wires, dev), n)
        else:
            raise ValueError(g.name)
    return state.reshape(batch, -1)

def expval_z(circuit: Circuit, angles: torch.Tensor) -> torch.Tensor:
    """<Z_k> для каждого кубита, форма (B, n)."""
    p = probabilities(run_circuit(circuit, angles))
    return p @ z_signs(circuit.n_qubits, p.device)
```

Приложение 3

Правило сдвига параметра и энкодер гибридной модели
(qubik/backends.py, qubik/models.py):

```
class _ParameterShift(torch.autograd.Function):
    """Градиент по правилу сдвига параметра (parameter-shift rule).

    Для вентилей вида  $\exp(-i \theta P / 2)$ , где  $P$  — оператор Паули:
     $d\langle O \rangle / d\theta = ( \langle O \rangle(\theta + \pi/2) - \langle O \rangle(\theta - \pi/2) ) / 2$ .
    Правило даёт точный градиент и применимо на реальном квантовом
    устройстве, где backprop через амплитуды невозможен.
    """

    @staticmethod
    def forward(ctx, angles, circuit, shots):
        ctx.circuit, ctx.shots = circuit, shots
        ctx.save_for_backward(angles)
        with torch.no_grad():
            ev = sim.expval_z(circuit, angles)
            if shots:
                ev = sim.sample_expval(ev, shots)
        return ev

    @staticmethod
    def backward(ctx, grad_out):
        (angles,) = ctx.saved_tensors
        circuit, shots = ctx.circuit, ctx.shots
        b, g = angles.shape
        shift = (math.pi / 2) * torch.eye(g, device=angles.device,
dtype=angles.dtype)
        with torch.no_grad():
            shifted = torch.cat(
                [angles[:, None, :] + shift, angles[:, None, :] - shift],
dim=1
            ).reshape(b * 2 * g, g)
            ev = sim.expval_z(circuit, shifted)
            if shots:
                ev = sim.sample_expval(ev, shots)
            ev = ev.reshape(b, 2, g, -1)
            d_ev = (ev[:, 0] - ev[:, 1]) / 2 # (B, G, n)
            grad_angles = torch.einsum("bgn,bn->bg", d_ev, grad_out)
        return grad_angles, None, None

def make_encoder(in_dim: int, n_qubits: int, enc: str = "pi2") ->
nn.Sequential:
    if enc == "pi":
        return nn.Sequential(nn.Linear(in_dim, n_qubits), ScaledTanh(1.0))
    if enc == "pi2":
        return nn.Sequential(nn.Linear(in_dim, n_qubits), ScaledTanh(0.5))
    if enc == "bn":
        return nn.Sequential(nn.Linear(in_dim, n_qubits),
nn.BatchNorm1d(n_qubits), ScaledTanh(0.5))
    raise ValueError(f"Неизвестный encoder: {enc}")
```